

Generative AI Essentials

Exploring Generative Pre-trained Transformers through Hands-On Implementation

Ayleen Santander

Assignment 13 — Module 11

Business Intelligence and Data Analytics Diploma

Willis College Online

September 16, 2026

Dataset: Project Gutenberg (<https://www.gutenberg.org/>)

1. Introduction to Generative AI and Its Significance

Generative artificial intelligence refers to a class of models that produce new content rather than merely classifying or predicting labels from existing content. Where a traditional supervised model answers a closed question, is this email spam, will this customer churn, a generative model answers an open one: what plausibly comes next. The output may be text, images, audio, code, or structured data, and it is synthesized rather than retrieved.

The distinction matters technically as well as conceptually. A discriminative model learns the boundary between classes, $P(y|x)$. A generative model learns the distribution of the data itself, and for language models this takes the specific form of learning $P(\text{next token} \mid \text{all previous tokens})$. Once that distribution is learned, text can be produced by sampling from it repeatedly, each new token becoming part of the context for the one after it. This autoregressive loop is the entire generation mechanism.

The significance of this shift is largely economic. Because the training objective, predict the next token, requires no human labelling, any text corpus becomes training data. This removed the annotation bottleneck that had constrained supervised NLP for decades and made it feasible to train on hundreds of billions of tokens. Scale then produced capabilities that were not explicitly designed for: models trained only to continue text turned out to be able to translate, summarize, answer questions, and write code, simply because those tasks appear as patterns within natural text.

For business intelligence and data analytics specifically, generative AI has moved from novelty to working tool in a short period. Practical uses now include natural-language querying of databases, automated narrative summaries accompanying dashboards, synthetic data generation for testing pipelines without exposing real records, code assistance for SQL and Python, and first-draft documentation of data models. Understanding how these systems work internally, rather than treating them as opaque services, is what allows an analyst to judge when their output can be trusted and when it cannot.

This report documents a hands-on investigation of that question. Using the pre-trained GPT-2 model and text from Project Gutenberg, it examines the architecture of generative pre-trained transformers, measures their behavior empirically, and implements two text generation models from scratch for comparison.

2. Description of GPT Architecture and Its Functionality

2.1 The transformer model

GPT models are built on a decoder-only transformer. The original 2017 transformer architecture contained two halves, an encoder that read the input sequence and a decoder that produced the output, because it was designed for machine translation. GPT discards the encoder entirely and retains only the decoder stack, which is what makes it a text generator rather than a text converter.

Text entering the model passes through several stages in sequence. First, a tokenizer splits it into sub word units using Byte Pair Encoding, drawing on the vocabulary and merge rules stored in the vocab.json and merges.txt files downloaded with the model. Each resulting token ID is then mapped to a dense

vector, and a positional embedding is added, since self-attention has no inherent notion of word order. These vectors flow through a stack of identical decoder blocks, each containing masked multi-head self-attention followed by a position-wise feed-forward network. A final linear projection converts the last hidden state into a probability distribution across the entire vocabulary.

Inspecting the loaded GPT-2 model directly confirms these dimensions:

```
cfg = model.config
print(f"Decoder blocks (layers): {cfg.n_layer}")
print(f"Attention heads per layer: {cfg.n_head}")
print(f"Embedding dimension: {cfg.n_embd}")
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
```

Architectural component	GPT-2 small value
Decoder blocks (layers)	12
Attention heads per layer	12
Embedding dimension	768
Dimensions per head	64
Maximum context length	1,024 tokens
Vocabulary size	50,257 tokens
Total parameters	124,439,808

Table 1. Architectural configuration of GPT-2 small, read from model.config.

The parameter counts divides unevenly across the components, which is itself informative:

Component	Parameters	Share of total
Feed-forward networks	56,669,184	45.5%
Embeddings (token + position)	39,383,808	31.6%
Attention	28,348,416	22.8%

Table 2. Distribution of parameters across model components.

Attention receives most of the conceptual attention in literature, yet the feed-forward layers hold roughly twice as many parameters. Current interpretability research suggests these layers function as a form of key-value memory storing factual associations, while attention handles the routing of information between positions.

2.2 Tokenization

Byte Pair Encoding sits between character-level and word-level tokenization. Frequent words remain single tokens while rare words decompose into sub word pieces, which allows a fixed 50,257-token vocabulary to represent any input without ever producing an unknown token. Running the tokenizer on a

sample sentence from *Pride and Prejudice* produced 17 tokens from 89 characters, a ratio close to the widely cited average of roughly four characters per token for English prose. The 'Ġ' prefix visible in the token output marks a preceding space, which is how the tokenizer preserves word boundaries without a separate delimiter.

2.3 Attention mechanisms

Self-attention allows every token to weigh the relevance of every other token in its context. For each position, three vectors are derived from the embedding using learned weight matrices: a Query representing what this token is looking for, a Key representing what it offers to others, and a Value representing the content it contributes if attended to. Attention scores are computed as the dot product of queries against keys, scaled by the square root of the key dimension to keep gradients stable, then normalized through a softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Two features are specific to GPT. The first is causal masking: before the softmax is applied, all positions to the right of the current token are set to negative infinity, so their weights become exactly zero. A token can therefore never attend to its own future, which is what makes the model autoregressive and suitable for generation. BERT omits this mask and reads bidirectionally, which is why it excels at classification but cannot generate text left to right.

The second is multi-head attention. Rather than performing one attention operation across all 768 dimensions, the model splits them across 12 heads of 64 dimensions each, computes attention in parallel within each, and concatenates the results. Different heads specialize during training, some track syntactic dependencies, others resolve references, others capture positional regularities.

Both properties were verified empirically. Extracting attention weights required reloading the model with the eager implementation, since the default fused kernel never materializes the attention matrix:

```
model = AutoModelForCausalLM.from_pretrained(
    "gpt2", attn_implementation="eager") # exposes attention weights
attn_out = model(**inp, output_attentions=True)
print(len(attn_out.attentions), attn_out.attentions[0].shape)
# 12 torch.Size([1, 12, 6, 6])
```

The returned tuple contained one tensor per layer, each of shape (batch, heads, sequence, sequence). Every row of every attention matrix summed to 1.0, confirming correct softmax normalization, and the entire upper triangle was zero, confirming the causal mask. Heatmaps of layers 1, 6, and 12 for the sentence "The cat sat on the mat" showed a clear progression: early layers concentrated almost all weight on the first token, while by layer 12 the distribution had spread across the available context, for example the final token distributed weight across all six positions rather than collapsing onto one.

The mechanism was then reproduced manually from the raw weight matrices to confirm understanding rather than rely on the library:

```
qkv = block.attn.c_attn(h) # 768 -> 2304
q, k, v = qkv.split(768, dim=2)
```

```
scores = (q1 @ k1.T) / math.sqrt(d_k)  # scaled dot product
mask = torch.triu(torch.ones_like(scores), diagonal=1).bool()
scores = scores.masked_fill(mask, float("-inf"))  # causal mask
weights = torch.softmax(scores, dim=-1)
output = weights @ v1
```

The resulting 6×6 weight matrix was lower-triangular as expected, with an output shape of (6, 64) corresponding to six tokens by one head's dimension.

2.4 How these models are trained

Pretraining is self-supervised. The objective is to predict the next token given all preceding tokens, so the training label is simply the input sequence shifted one position to the right. No human annotation is required, which is what permits training on corpora of hundreds of billions of tokens. The model is scored using cross-entropy loss between its predicted distribution and the actual next token, and weights are updated through backpropagation using the Adam optimizer.

This objective is the source of the name: Generative because it produces text, Pre-trained because it learns general language structure before any task-specific adaptation, and Transformer for the underlying architecture.

Pretraining alone produces a model that continues text plausibly but does not follow instructions. Modern deployed assistants add two further stages: supervised fine-tuning on curated instruction-response pairs, then reinforcement learning from human feedback, in which human rankings train a reward model that guides policy optimization. This alignment stage explains the difference between base GPT-2, which tends to drift off topic, and a production assistant that answers what was asked.

Scale is itself a defining characteristic of the family. GPT-1 contained 117 million parameters, GPT-2 1.5 billion, and GPT-3 175 billion, with the architecture largely unchanged across those generations. Capabilities such as few-shot learning emerged from scale rather than from structural innovation.

3. Methodology and Findings from the Hands-On Implementation

3.1 Dataset and environment

The corpus was drawn from Project Gutenberg (<https://www.gutenberg.org/>), a public-domain digital library. Two acquisition methods were used. Direct HTTP retrieval fetched *Pride and Prejudice* (book ID 1342) and stripped the license boilerplate with regular expressions, yielding 744,279 characters. The NLTK Gutenberg corpus provided an offline alternative, from which *Emma* was loaded at 887,071 characters. The offline route avoids the rate limiting that Project Gutenberg applies to repeated requests from a single address.

All work was carried out in Google Colab using PyTorch 2.11 and Hugging Face Transformers 5.16, with GPT-2 small as the pre-trained reference model.

3.2 Measuring the training objective

Because cross-entropy loss is the exact quantity minimized during pretraining, evaluating it on held-out text measures directly what pretraining achieved. Perplexity, the exponential of that loss, expresses the same information as an effective branching factor: how many equally likely options the model is choosing among at each position.

```
def perplexity(txt):
    enc = tokenizer(txt, return_tensors="pt", truncation=True, max_length=1024)
    with torch.no_grad():
        return torch.exp(model(**enc, labels=enc["input_ids"]).loss).item()
```

Input text	Perplexity	Interpretation
Emma (Jane Austen prose)	36.18	Familiar register, well predicted
Pride and Prejudice excerpt	257.74	Higher on a dialogue-heavy passage
Shuffled nonsense strings	899.80	No learnable structure
Contemporary slang	1,332.91	Post-dates the 2019 training corpus

Table 3. Perplexity of GPT-2 across text types. Lower values indicate better prediction.

The roughly 37-fold spread between Austen prose and contemporary slang is the clearest available evidence of what pretraining encoded. The model has internalized nineteenth-century literary English, which is heavily represented in its training data, while vocabulary coined after 2019 falls outside what it learned. This also illustrates a practical limitation worth noting for any deployment: a language model's competence is bounded by the distribution of its training corpus, and performance degrades on text drawn from outside it.

The prediction mechanism was then inspected token by token on the sentence "The cat sat on the mat", which produced a loss of 4.8504 and perplexity of 127.79. Examining each position revealed the model predicting "was" where the actual next token was "sat", "on" correctly after "sat", "the" correctly after "on", and "floor" where the actual token was "mat". Each of these near-misses is grammatically valid, which is precisely what next-token prediction optimizes for, plausibility rather than factual match.

3.3 Generation behavior and the temperature parameter

Text was generated with nucleus sampling at three temperature settings. Temperature rescales the logits before the softmax: values below 1 sharpen the distribution toward the highest-probability token, values above 1 flatten it toward uniform.

Temperature	Observed behavior
0.3	Highly repetitive; phrases and whole clauses recycled within a few sentences
0.8	Coherent sentence structure with reasonable topical variety
1.3	Unpredictable topic shifts and weakening grammatical coherence

Table 4. Effect of sampling temperature on generated output.

A notable finding emerged from the prompt "It is a truth universally acknowledged," the opening of *Pride and Prejudice*. Rather than continuing the novel, GPT-2 produced text about government monetary policy, economic conflict, and academic citation formats. Despite the novel being present in its training corpus, the model did not reproduce it. This demonstrates that pretraining learns a probability distribution over language rather than memorizing and retrieving documents, and it illustrates concretely why a base model without instruction tuning is unsuitable for direct deployment.

3.4 Model 1 — character-level LSTM

To contrast the pre-trained transformer against a model trained from scratch, a character-level recurrent network was implemented in PyTorch. Training used 200,000 characters of *Pride and Prejudice* over a vocabulary of 89 distinct characters, with inputs and targets offset by one position to replicate the next-token objective at character granularity.

Property	Value
Architecture	2-layer LSTM, 128-dim embedding, 256 hidden units
Parameters	955,865
Training steps	500
Initial loss (train / val)	1.989 / 2.240
Final loss (train / val)	1.265 / 1.326

Table 5. Character-level LSTM configuration and results.

Training and validation curves descended together with only a small gap between them, indicating that the model had not yet overfitted and would benefit from further training. Generated output at temperature 0.4 produced correctly spelled English words, appropriate capitalization after sentence boundaries, correctly paired quotation marks, and character names drawn from the source text, but no semantic coherence across clauses. This is the expected ceiling for a one-million-parameter model trained on 200,000 characters.

3.5 Model 2 — mini-GPT transformer

A second model reimplemented the transformer architecture from first principles, so that every component described in Section 2 appears explicitly in code rather than being hidden inside a library call.

Property	Value
Architecture	4 decoder blocks, 6 attention heads, 192-dim embedding
Context window	128 characters
Parameters	1,836,377
Training steps	3,000
Initial loss (train / val)	4.715 / 4.476

Property	Value
Final loss (train / val)	1.209 / 1.370

Table 6. Mini-GPT configuration and results.

Each architectural claim from Section 2 maps to a specific line of this implementation. The scaled dot product appears as `q @ k.transpose(-2, -1) * k.shape[-1] ** -0.5`. The causal mask appears as `masked_fill(self.mask[:T, :T] == 0, float("-inf"))` applied against a lower-triangular buffer. Residual connections appear as `x = x + self.attn(self.ln1(x))`, with layer normalization applied before each sub-layer. The feed-forward expansion to four times the embedding dimension appears as `nn.Linear(N_EMB, 4 * N_EMB)`. The training objective appears as `cross_entropy` computed against targets shifted by one position.

The mini-GPT reached a final training loss of 1.209 against the LSTM's 1.265, and did so from a much higher starting point, 4.715 versus 1.989, reflecting its random initialization across a larger parameter space. Generated output showed a qualitative improvement over the LSTM: recognizable dialogue structure with correctly paired quotation marks spanning multiple lines, character names used in grammatically appropriate positions, and period-appropriate sentence rhythm. Semantic content remained incoherent, as expected at this scale.

The comparison is instructive because it isolates architecture from scale. Both models saw identical data with an identical objective; the transformer achieved better loss because self-attention gives every position direct access to all previous positions, whereas the LSTM must compress all prior context into a fixed-size hidden state that is updated sequentially. This advantage is the practical reason transformers displaced recurrent networks for language modelling.

4. Applications of Generative AI and Practical Demonstration

4.1 Application domains

Generative language models have found production use across several categories of work:

- Content creation = drafting marketing copy, product descriptions, reports, and email at volumes that would be impractical manually.
- Software development = code completion, test generation, documentation, and translation between programming languages.
- Data analysis= natural-language querying of databases, automated narrative commentary on dashboards, and generation of synthetic datasets for testing pipelines without exposing real records.
- Customer support = first-line response drafting, ticket summarization, and multilingual translation.
- Education= explanation at adjustable levels of difficulty, practice question generation, and feedback on written work.
- Accessibility= text simplification, summarization of long documents, and format conversion for assistive technologies.

4.2 Practical demonstration

The mini-GPT trained in Section 3.5 was used as a content-creation tool, generating period-styled prose from a seed phrase. The generation function crops the context to the model's 128-character window, applies temperature scaling to the final position's logits, samples one character, appends it, and repeats:

```
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=0.8):
    for _ in range(max_new_tokens):
        logits, _ = self(idx[:, -SEQ_LEN:])          # crop to context window
        probs = F.softmax(logits[:, -1, :] / temperature, dim=-1)
        idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1)
    return idx
```

Seeded with "It is a truth", the model produced 400 characters of output containing correctly formed dialogue with paired quotation marks, character names from the source novel used in plausible grammatical positions, and sentence structures consistent with the period. Individual clauses did not carry meaning.

This outcome is itself the useful finding for a content-creation application. A model of this size can reliably reproduce the surface characteristics of a writing style, register, punctuation conventions, rhythm, vocabulary distribution, but cannot produce content that means anything. The practical implication is that style transfer and formatting are achievable at small scale, while any application requiring factual or logical content demands either a far larger model or retrieval of verified source material. Both models in this assignment reinforce the same conclusion reached by the GPT-2 experiments: fluency and reliability are separate properties, and fluency arrives first.

5. Ethical Considerations and Potential Solutions

5.1 Factual unreliability

Because the training objective rewards plausibility rather than truth, these models produce confident, well-formed text regardless of whether the content is accurate. The temperature experiments in this assignment illustrated the mechanism directly: GPT-2 generated fabricated academic citations, complete with author names, journal titles, volume numbers, and page ranges, none of which exist. The output was formatted convincingly enough to pass casual inspection.

Mitigations include retrieval-augmented generation, in which the model is supplied with verified source documents and instructed to draw only on them; citation requirements that link claims to checkable sources; and human review for any output with material consequences. None of these eliminates the problem, and no purely technical fix is currently available.

5.2 Bias in training data

Models learn the statistical patterns of their training corpus, including its prejudices. GPT-2 was trained largely on web text scraped in 2019, which over-represents some demographics and viewpoints and under-represents others. The perplexity measurements in Section 3.2 demonstrated this quantitatively

from a different angle: text outside the training distribution, in that case contemporary slang, was assigned dramatically lower probability. The same mechanism applies to dialects, regional varieties, and non-dominant registers, meaning the model performs measurably worse for some speaker populations than others.

Mitigations include curation and auditing of training corpora, evaluation disaggregated across demographic groups rather than reported only in aggregate, documentation through model cards that states known limitations explicitly, and diverse review teams during development.

5.3 Copyright and consent

This assignment deliberately used Project Gutenberg, whose texts are in the public domain. Most commercial models were trained on material scraped without the consent of authors or rights holders, and litigation on this question is ongoing. The *Pride and Prejudice* experiment showed that GPT-2 had clearly been exposed to the novel yet did not reproduce it, models learn distributions rather than storing documents. This complicates the legal analysis considerably but does not resolve the underlying question of whether training constitutes fair use.

Mitigations include licensed or public-domain training data, opt-out mechanisms for rights holders, provenance documentation, and compensation frameworks for creators whose work contributes to training.

5.4 Misinformation and misuse

The same capability that makes these models useful for content creation makes them useful for generating propaganda, spam, phishing messages, and fabricated reviews at scale and at negligible cost. Detection is unreliable, and existing detectors produce false positives that have caused real harm in academic settings.

Mitigations include provenance watermarking, platform-level rate limiting, usage policies with enforcement, and public literacy about synthetic content. The economics currently favors the attacker, since generation is cheaper than verification.

5.5 Environmental and labour costs

Training large models consumes substantial energy, and the alignment stages described in Section 2.4 depend on human annotators who review content, including harmful material, often under poor conditions and low pay.

Mitigations include efficiency research such as distillation and quantization, transparent reporting of compute and emissions, carbon-aware scheduling of training runs, and fair labour standards for annotation work.

5.6 Summary of mitigations

Concern	Primary mitigation
Factual unreliability	Retrieval grounding, citation requirements, human review
Training data bias	Corpus auditing, disaggregated evaluation, model cards
Copyright and consent	Licensed data, opt-out mechanisms, provenance records
Misinformation and misuse	Watermarking, rate limits, enforcement, public literacy
Environmental and labour cost	Efficiency research, transparent reporting, fair labour standards

Table 7. Principal ethical concerns and corresponding mitigations.

A theme runs through all five: none is solved by better engineering alone. Each requires institutional choices about what to build, what to train on, and what to deploy.

6. Conclusion and Future Perspectives

This assignment examined generative pre-trained transformers from three angles, architectural inspection, empirical measurement, and implementation from scratch, using text from Project Gutenberg throughout.

The architectural findings confirmed the decoder-only design: 12 layers, 12 attention heads, 768 embedding dimensions, and 124 million parameters in GPT-2 small, with the parameter count distributed unevenly such that feed-forward layers hold roughly twice the weights of attention. Attention weights extracted from the model confirmed both defining properties of causal self-attention, rows summing to 1.0 after softmax, and a zeroed upper triangle enforcing that no token attends to its future.

The empirical findings were arguably more instructive than the architectural ones. Perplexity varying by a factor of 37 across text types measured directly what pretraining encoded and where it fails. Token-level inspection showed the model predicting grammatically valid alternatives rather than exact matches, which is what next-token prediction optimizes for. Most tellingly, prompting with the opening line of *Pride and Prejudice* produced text about monetary policy rather than the novel, a concrete demonstration that these models learn distributions rather than storing and retrieving documents.

The implementation findings isolated architecture from scale. Trained on identical data with an identical objective, the mini-GPT reached a lower final loss than the LSTM (1.209 against 1.265) despite starting from a far worse initialization and produced qualitatively better-structured output. Neither model produced meaningful content, which locates the boundary clearly: fluency is achievable at one to two million parameters, while meaning is not.

Several directions appear likely to shape the field near term. Efficiency is receiving as much attention as raw scale, through distillation, quantization, and mixture-of-experts architectures that activate only part of the network per token. Retrieval augmentation is becoming standard practice, grounding generation in verified documents rather than relying on parametric memory, a direct response to the unreliability documented in Section 5.1. Multimodal models that process text, images, and audio within one architecture are becoming the default rather than the exception. Small, specialized models fine-tuned for

narrow domains are proving competitive with much larger general models at a fraction of the deployment cost, which matters for organizations that cannot run frontier-scale infrastructure.

For analytics work specifically, the practical conclusion from this assignment is that these systems are best treated as drafting and exploration tools operating under review, rather than as authorities. A model that generates fabricated citations in convincing academic format, and that continues a canonical novel with unrelated economic commentary, is demonstrably capable of producing fluent text that is entirely wrong. Understanding the mechanism that produces that behaviour, next-token prediction over a learned distribution, with no internal representation of truth, is what allows an analyst to use the capability productively while designing appropriate verification around it.

References

Project Gutenberg. (n.d.). Free eBooks. <https://www.gutenberg.org/>